Ders 23 Genişletilmiş Notu: Anahtar İndeksli Sayma, LSD Dizgi Sıralama ve Vize Sorularının Çözümü

Ders 23 Genişletilmiş Notu: Anahtar İndeksli Sayma, LSD Dizgi Sıralama ve Vize Sorularının Çözümü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dizgi Sıralamaya Giriş: Sayılar ve Dizgiler
- 2. Anahtar İndeksli Sayma (Key-Indexed Counting)
- 3. LSD Dizgi Sıralama (Least Significant Digit First String Sort)
- 4. Vize Sorularının Çözümü
- 5. Programlama Sorusu: Arama Motoru Bağlı Listesi ve Tasarım Hataları
- Örnek Sorular
- Dersin Toparlaması

1. Dizgi Sıralamaya Giriş: Sayılar ve Dizgiler

Bu ders dönemin son dersidir. Hoca dersi iki parçaya ayırdı: önce dizgi (string) sıralama algoritmalarını anlattı, sonra vize sorularından üzerinde en çok takılınanları çözdü.

1.1 Sayı Karşılaştırması ile Dizgi Karşılaştırmasının Farkı

Şimdiye kadar gördüğümüz sıralamalarda elimizde sayılar vardı ve her sayıyı bir bütün olarak değerlendiriyorduk. Örneğin 34'ü 45 ile, 4579'u 3251 ile tek bir karşılaştırmada kıyaslayabiliyoruz.

Dizgilerde durum farklıdır. "kalem" ya da "mavi" gibi bir dizgi, her biri ayrı bir değer taşıyan karakterlerden oluşur: `k`, `a`, `l`, `e`, `m`. Bilgisayarda her karakterin bir sayı karşılığı vardır, ama dizginin tamamı tek bir sayı gibi ele alınamaz. Bu yüzden iki dizgi, karakterleri soldan sağa tek tek karşılaştırılarak kıyaslanır.

Hocanın örneği "kalem" ile "kel" karşılaştırmasıdır. İlk karakterler (`k`) aynıdır. İkinci karakterde `a` alfabede `e`'den önce geldiği için `a < e` olur. Karşılaştırma burada biter ve "kalem" < "kel" sonucuna varılır. Yani bir karakterin diğerinden büyük olduğu görüldüğü anda karşılaştırma bırakılır ve dizgilerin birbirine göre durumuna karar verilir. Hoca, bu mantığı gözünüzde canlandırmak için sözlüğü düşünmenizi önerdi: sözlükteki sıralama aynen budur. "kaz" ile "kaza" da aynı şekilde karakter karakter karşılaştırılmak zorundadır.

1.2 Sıralama Algoritmalarının Karmaşıklıklarının Hatırlatması

Hoca, dizgilere geçmeden önce sayılar üzerinde gördüğümüz sıralama algoritmalarının karmaşıklıklarını hatırlattı:

- Seçmeli sıralama (selection sort), kabarcık sıralama (bubble sort) ve araya sokmalı sıralama (insertion sort): en kötü durumda (worst case) $O(n^2)$.
- Birleştirmeli sıralama (merge sort) ve yığın sıralama (heap sort): en kötü durumda $O(n \log n)$.

- Hızlı sıralama (quicksort): en kötü durumda $O(n^2)$, ortalama durumda (average case) $O(n \log n)$.

Quicksort ortalama durumu $O(n \log n)$ olduğu için en çok tercih edilen algoritmadır. Çalışma prensibi diziyi ikiye bölerek ilerlemektir. Dizi başlangıçta sıralı verilirse bu bölme dengeli yapılamaz ve karmaşıklık $O(n^2)$'ye çıkar. Ama sıralı bile olsa verilen dizi önce karıştırılırsa (shuffle) karmaşıklık $O(n \log n)$'ye indirilmiş olur. Hoca bu konunun gelecek dönem Algoritma Analizi dersinde ayrıntılı görüleceğini söyledi.

> **Ders sonrası not:** Hoca bu sırada "average case'lerin hepsi aynı" dedi. Bu ifade merge sort, heap sort ve quicksort için geçerlidir; üçünün de ortalama durumu $O(n \log n)$'dir. Selection, bubble ve insertion sort'un ortalama durumu ise $O(n^2)$'dir.

Sayıları bir bütün olarak karşılaştırabiliyorken dizgileri karakter karakter karşılaştırmak zorunda olduğumuza göre, dizgileri nasıl sıralayacağız? Hoca bu derste bunun için LSD (least significant digit first, en düşük anlamlı basamaktan başlayan) dizgi sıralamayı anlatacağını, MSD (most significant digit first, en yüksek anlamlı basamaktan başlayan) dizgi sıralamayı ise öğrencilere inceleme için bırakacağını söyledi.

2. Anahtar İndeksli Sayma (Key-Indexed Counting)

LSD sıralamanın temel taşı anahtar indeksli saymadır. Bu yöntem, sayı olmayan sembol bilgilerini basamak basamak değerlendirir ve sembollerin kendisini dizi indisi (adres) olarak kullanır. Her seferinde tek bir basamak (digit) sıralanır. Hoca önce tek basamaklı bir örnekle, yani tek karakterlerden oluşan bir diziyle başladı.

2.1 Varsayım: Anahtarlar Dizi İndisi Olarak Kullanılabilir

Anahtarların (key) 0 ile $R-1$ arasında tam sayılar olduğunu ve bu anahtarları doğrudan dizi adresi olarak kullanabildiğimizi varsayıyoruz. Burada R sembol sayısıdır (number of symbols). Örnekteki semboller `a`, `b`, `c`, `d`, `e`, `f` olduğu için $R = 6$'dır. Küçük harflerin tamamı kullanılsaydı $R = 26$ olurdu.

Karakterleri sayıya çevirmek için C'nin bir özelliğinden yararlanıyoruz. C'de `char` tipindeki değişkenler istendiğinde sayı gibi kullanılabilir, çünkü bilgisayar ortamında her şey zaten sayı olarak saklanır. Hoca bunu uydurma bir kodla anlattı: `a` karakterinin kodu 80, `x`'in kodu 90 olsaydı `x - 'a'` işlemi doğrudan $90 - 80$ işlemini yapardı. Aynı mantıkla:

- `'a' - 'a'` $= 0$,
- `'c' - 'a'` $= 2$.

Yani `karakter - 'a'` ifadesi, her harfi 0'dan başlayan bir indise çevirir. Yöntemin tamamı bu fikre dayanır.

2.2 Örnek Dizi ve Temel Sezgi

Hoca şu 12 elemanlı karakter dizisini kullandı:

```
indis:  0  1  2  3  4  5  6  7  8  9  10 11
a:      d  a  c  f  f  b  d  b  f  b  e  a
```

Hoca önce şunu sordu: Bu dizi sıralı olsaydı nasıl görünürdü?

```
indis:  0  1  2  3  4  5  6  7  8  9  10 11
sıralı: a  a  b  b  b  c  d  d  e  f  f  f
```

Sıralı halde `a` 0. indisten, `b` 2'den, `c` 5'ten, `d` 6'dan, `e` 8'den, `f` 9'dan başlar ve dizi 12'de biter. Eğer her harften kaç tane olduğunu sayabilirsek, her harfin sıralı dizide hangi indisten başlayacağını da bulmuş oluruz. Algoritmanın bütün mantığı budur.

2.3 Adım 1: Frekansları Sayma

$R + 1$ elemanlı bir `count` dizisi kullanılır. Hoca burada küçük bir hile yaptı: her harfin sayısını kendi gözüne değil, bir sonraki göze yazdı. Yani `a`'ların sayısı `count[1]`'e, `b`'lerin sayısı

`count[2]`'ye yazılır. `count[0]` ise 0 kalır. Bunun nedeni bir sonraki adımda görülecek: `a` zaten 0'dan başlar, `a`'ların sayısı ise `b`'nin nereden başlayacağını gösterir.

```
for (i = 0; i < N; i++)          /* N: a dizisinin eleman sayısı */
    count[(a[i] - 'a') + 1]++;
```

Hoca buradaki parantezlerin gerekmediğini, yalnız gözle rahat ayırt edilsin diye koyduğunu söyledi. Dizi baştan sona gezilince sayılar şöyle olur:

Harf	a	b	c	d	e	f
Sayı	2	3	1	2	1	3

Hoca sayarken birkaç kez karıştırıp düzeltti; doğru değerler tablodaki gibidir (toplam 12).

2.4 `count` Dizisini Sıfırlamak

Hoca, `count` dizisinin sayma başlamadan önce mutlaka sıfırlanması gerektiğini vurguladı. Sıfırlanmazsa bellekte o anda hangi değerler varsa onların üzerine eklenerek devam edilir ve sonuç bozulur. Sıfırlamanın üç yolunu gösterdi:

```
int count[10] = {0};                        /* tanım satırında sıfırlama */
int *count = calloc(R + 1, sizeof(int));    /* calloc belleği sıfırlar */
for (r = 0; r <= R; r++) count[r] = 0;      /* döngüyle sıfırlama */
```

- Tanım satırında ilk göze 0 yazıldığında geri kalan gözlere de derleyici 0 koyar. Sembol sayısı önceden biliniyorsa bu yol kullanılabilir.
- Sembol sayısı önceden bilinmiyorsa `count` işaretçi olarak tanımlanır ve bellek ayrılır. `malloc` belleği sıfırlamaz; `malloc` yerine `calloc` kullanılırsa ayrılan bellek sıfırlanmış olarak gelir.
- Normal dizi tanımlanıp bir döngüyle de sıfırlanabilir.

  Hocanın vurgusu: Sıfırlamaya dikkat etmelisiniz. Sıfırlamazsanız bellekte hangi değerler varsa onların üzerine ekleyerek devam eder.

2.5 Adım 2: Frekansları Başlangıç İndislerine Çevirme

Asıl bilmek istediğimiz şey her harften kaç tane olduğu değil, sıralı dizide her harfin nereden başlayacağıdır. Bunun için frekanslar kümülatif olarak toplanır:

```
for (r = 0; r < R; r++)
    count[r + 1] += count[r];
```

Hoca bunu tek tek söyleyerek kurdu:

- `a` 0'dan başlar.
- İki tane `a` olduğu için `b` 2'den başlar.
- Üç tane `b` ve iki tane `a` olduğu için `c` $2 + 3 = 5$'ten başlar.
- Bir tane `c` olduğu için `d` 6'dan başlar.
- İki tane `d` olduğu için `e` 8'den başlar.
- Bir tane `e` olduğu için `f` 9'dan başlar.
- Üç tane `f` olduğu için dizi 12'de biter.

Harf	a	b	c	d	e	f	(son)
`count` (başlangıç)	0	2	5	6	8	9	12

Son göz (12) algoritmada kullanılmaz. Hoca, bu ek gözü anlaşılırlık için kullandığını, istenirse ekstra göz kullanmadan doğrudan `count[r]` üzerinde oynayarak da aynı işin yapılabileceğini söyledi.

2.6 Adım 3: Elemanları Geçici Diziye Dağıtma

Sonucu yine `a` dizisinde saklamak istediğimiz için, aynı boyutta geçici (temporary) bir `tmp` dizisi tanımlanır. Önce sıralı hal `tmp`'ye yazılır, sonra `tmp` geri `a`'ya kopyalanır.

`a` dizisi baştan sona gezilir. Her karakter için `count` dizisindeki başlangıç indisine bakılır, karakter `tmp`'nin o gözüne yazılır ve o başlangıç indisi bir artırılır. Artırmanın anlamı şudur: aynı harften bir sonraki gelen artık bir sonraki göze yerleşecektir.

```
for (i = 0; i < N; i++) {
    tmp[count[a[i] - 'a']] = a[i];
    count[a[i] - 'a']++;
}
```

Hocanın tahtada yaptığı iz adım adım şöyledir:

Adım	`a[i]`	Yazıldığı `tmp` gözü	Güncellenen `count`
0	d	6	d: 6 → 7
1	a	0	a: 0 → 1
2	c	5	c: 5 → 6
3	f	9	f: 9 → 10
4	f	10	f: 10 → 11
5	b	2	b: 2 → 3
6	d	7	d: 7 → 8
7	b	3	b: 3 → 4
8	f	11	f: 11 → 12
9	b	4	b: 4 → 5
10	e	8	e: 8 → 9
11	a	1	a: 1 → 2

Hoca ilk `d` için şunu söyledi: "D diyor ki 6'dan başlayacak. 6. göze D'yi yerleştirdim. Bu ne demek? Bir sonra gelen D artık 7. göze yerleşecek." Tek bir `c` olsa bile `c`'nin sayacını 6'ya çıkarmak gerekir; hoca bunu kafa karışmasın diye her zaman yapmak gerektiğini söyledi. Dizi bitince `tmp` şöyledir:

```
tmp:    a  a  b  b  b  c  d  d  e  f  f  f
```

2.7 Adım 4: Geri Kopyalama ve Tam Program

Son olarak `tmp` dizisi `a` dizisine geri kopyalanır:

```
for (i = 0; i < N; i++)
    a[i] = tmp[i];
```

Hoca, sonra yazdırma gibi kısımların eklenebileceğini söyledi. Dört adımı bir araya getiren, derlenip çalıştırılmış tam program aşağıdadır. Fonksiyon adı ve parametreleri, hocanın anlattığı `void` dönen, `a` dizisini ve `N` eleman sayısını alan "key index sort" fonksiyonu tarifine göre yazıldı (ara adım eklendi).

```
#include <stdio.h>
#include <stdlib.h>

#define R 6   /* sembol sayısı: 'a'..'f' */

void keyIndexedSort(char a[], int N)
{
    int i, r;
    int count[R + 1] = {0};           /* mutlaka sıfırlanmalı */
    char *tmp = malloc(N * sizeof(char));

    for (i = 0; i < N; i++)           /* 1. frekansları say */
        count[(a[i] - 'a') + 1]++;
```

```
    for (r = 0; r < R; r++)            /* 2. başlangıç indisleri */
        count[r + 1] += count[r];
    for (i = 0; i < N; i++) {          /* 3. dağıt */
        tmp[count[a[i] - 'a']] = a[i];
        count[a[i] - 'a']++;
    }
    for (i = 0; i < N; i++)            /* 4. geri kopyala */
        a[i] = tmp[i];
    free(tmp);
}

int main(void)
{
    char a[] = {'d','a','c','f','f','b','d','b','f','b','e','a'};
    int i, N = sizeof(a) / sizeof(a[0]);
    keyIndexedSort(a, N);
    for (i = 0; i < N; i++)
        printf("%c ", a[i]);
    printf("\n");
    return 0;
}
```

Çıktı:

```
a a b b b c d d e f f f
```

2.8 Soru-Cevap: Kopyalamak Yerine İşaretçileri Değiştirmek

Soru: Algoritmayla ilgili değil ama, değerleri `tmp`'den `a`'ya tek tek kopyalamak yerine `a` ile `tmp`'nin gösterdiği adresleri değiştirsek yeterli olur mu?

Cevap: Diziler statik tanımlandıysa bu yapılamaz. Ama `a` ve `tmp` için bellek ayırma (memory allocation) ile yer tutulduysa işaretçiler değiştirilebilir. `a`'yı gösteren işaretçi artık `tmp`'nin başlangıç adresini gösterir, `a` için eskiden tutulan alan da `free` ile bırakılır. Böylece kopyalamadan aynı iş yapılmış olur. Hoca bunun kesinlikle doğru olduğunu, ama C'ye özgü bir ayrıntı olduğu için asıl önemli olanın mantığı anlamak olduğunu ekledi. İşaretçilerle uğraşırken doğru yapılırsa sorun yoktur, hatalı yapılırsa çok sıkıntı çıkar; bu yüzden çok dikkat etmek gerekir.

Hoca bu soru üzerine bir ayrıntı daha ekledi. Sıralama bir fonksiyon içinde yapılıyorsa ve `tmp` o fonksiyonun içinde `malloc` ile ayrılıp sonunda `free` ile bırakılıyorsa, fonksiyondan çıkıldığında `tmp` için yer tutulmaya devam edilmez; ana dizi olarak yine `a` kullanılmış olur. Aynı şey fonksiyon içinde yerel (local) tanımlanan bir dizi için de geçerlidir: örneğin fonksiyon içinde 100 elemanlı bir `tmp` tanımlanırsa, fonksiyondan çıkıldığında onun için tutulan yer bırakılır.

2.9 Yedek Dizi Kullanmak İyi mi, Kötü mü?

Hoca, gelecek dönem görülecek merge sort'ta da yedek dizi tutulduğunu hatırlatıp sınıfa şu soruyu sordu: Sıralama yaparken işlerin yedek bir dizi üzerinden yürütülmesi, optimizasyon ve etkinlik açısından iyi mi, kötü mü?

Bir öğrenci, yer karmaşıklığı (space complexity) açısından kötü olduğunu, ama zamana yardım ettiği için kendini dengelediğini söyledi. Hoca bu cevabı doğru buldu ve kararın duruma göre verilmesi gerektiğini anlattı:

- Bin ya da on bin elemanı merge sort veya yedek dizi kullanan başka bir yöntemle sıralamak pek zarar vermez.
- Eleman sayısı çok arttığında ise yedek bir dizi için büyük bir bellek alanı kullanmak yer kullanımında sorun yaratabilir.
- Bu yüzden birinci sınıftan beri dizinin kendisi üzerinde iş yapma çabası anlatılır. Selection sort, insertion sort ve quicksort gibi algoritmalarda işlem dizinin kendisi üzerinde yapılmaya çalışılır.

Anahtar indeksli sayma da, ondan türeyen LSD sıralama da yedek yer kullanan yöntemlerdir.

Hocanın vurgusu: Yer sizin için önemliyse, geçici yedek dizi kullanan algoritmalara dikkat etmeniz gerekir.

3. LSD Dizgi Sıralama (Least Significant Digit First String Sort)

LSD dizgi sıralama, az önce anahtar indeksli sayma ile tek karakter için yaptığımız işlemin dizgilerin bütün karakter konumları (basamakları) için tekrarlanmasıyla elde edilir. Bu derste dizgilerin hepsi aynı uzunluktadır (her biri 3 karakter).

3.1 Temel Fikir: Sağdan Sola ve Kararlı

Değerlendirme sağdan sola doğru yapılır. Adındaki "least significant digit first" ifadesinin söylediği gibi, sıralama en düşük anlamlı basamaktan, yani dizginin son karakterinden başlar. Bir sayı için düşünülürse, 379'da önce 9'a, sonra 7'ye, en son 3'e bakılır. Her turda bir önceki turun sıraladığı hal korunarak devam edilir.

Bu, kararlı (stable) bir sıralama yöntemidir. Kararlı sıralamada kastedilen şey, bir önceki adımın sonucunun bir sonraki adımda korunmasıdır. Daha açık söylersek: aynı değere sahip iki eleman varsa birbirlerine göre yerleri değişmez. Hocanın örneğiyle, bir listede 2. sırada bir "elma", 5. sırada başka bir "elma" varsa, sıralamadan sonra da 2. sıradaki elma diğerinin üstünde kalmaya devam eder. Hoca bunun neden gerekli olduğunun örnek üzerinde anlaşılacağını söyledi.

Anahtar indeksli sayma bu özelliği kendiliğinden sağlar: dizi baştan sona gezilir ve aynı harfe sahip elemanlar, geldikleri sırayla art arda gözlere yerleştirilir.

3.2 Örnek Dizi

Hoca, her biri 3 karakterli 12 dizgiden oluşan şu diziyi kullandı (`a`-`f` sembolleri, $R = 6$):

```
indis:  0    1    2    3    4    5    6    7    8    9    10   11
a:      dab  add  cab  fad  fee  bad  dad  bee  fed  bed  ebb  ace
```

3.3 1. Tur: Son (3.) Karaktere Göre Sıralama

Önce her dizginin son karakterine bakılır ve bu karakterler sayılır. Hoca saymaya başlarken bir kez yanlış göze yazıp düzeltti; doğru sayım şöyledir:

Harf	a	b	c	d	e	f
Sayı	0	3	0	6	3	0
Başlangıç	0	0	3	3	9	12

Yani 3 tane `b`, 6 tane `d`, 3 tane `e` vardır. `b` ile biten dizgiler 0'dan, `d` ile bitenler 3'ten, `e` ile bitenler 9'dan başlar. Mantık tek karakterli örnekle aynıdır: önce geçici `tmp` dizisine yazılır, sonra geri kopyalanır. Hocanın tahtadaki izi:

Gelen dizgi	Son karakter	`tmp` gözü	Güncellenen `count`
dab	b	0	b: 0 → 1
add	d	3	d: 3 → 4
cab	b	1	b: 1 → 2
fad	d	4	d: 4 → 5
fee	e	9	e: 9 → 10
bad	d	5	d: 5 → 6
dad	d	6	d: 6 → 7
bee	e	10	e: 10 → 11
fed	d	7	d: 7 → 8
bed	d	8	d: 8 → 9
ebb	b	2	b: 2 → 3

Gelen dizgi	Son karakter	`tmp` gözü	Güncellenen `count`
ace	e	11	e: 11 → 12

1. turun sonucu (son karaktere göre sıralı):

```
indis:  0    1    2    3    4    5    6    7    8    9    10   11
        dab  cab  ebb  add  fad  bad  dad  fed  bed  fee  bee  ace
```

3.4 2. Tur: Ortadaki (2.) Karaktere Göre Sıralama

Şimdi 1. turun sonucu üzerinde ikinci karakterlere bakılır. Hoca burada da önce yanlışlıkla birinci karakterleri saymaya başladı, sonra öğrencilerin kendisini uyarmadığından yakınarak baştan aldı. Doğru sayım:

Harf	a	b	c	d	e	f
Sayı	5	1	1	1	4	0
Başlangıç	0	5	6	7	8	12

Toplam 12’dir; `f` olmadığı için 12’den başlar ve dizi orada biter. Yerleştirme izi:

Gelen dizgi	2. karakter	`tmp` gözü	Güncellenen `count`
dab	a	0	a: 0 → 1
cab	a	1	a: 1 → 2
ebb	b	5	b: 5 → 6
add	d	7	d: 7 → 8
fad	a	2	a: 2 → 3
bad	a	3	a: 3 → 4
dad	a	4	a: 4 → 5
fed	e	8	e: 8 → 9
bed	e	9	e: 9 → 10
fee	e	10	e: 10 → 11
bee	e	11	e: 11 → 12
ace	c	6	c: 6 → 7

2. turun sonucu (son iki karaktere göre sıralı):

```
indis:  0    1    2    3    4    5    6    7    8    9    10   11
        dab  cab  fad  bad  dad  ebb  ace  add  fed  bed  fee  bee
```

3.5 Kararlılık Neden Önemli?

Hoca 2. turun sonucunda `cab` ile `fad`’e dikkat çekti. İkisinin de ikinci karakteri `a`’dır, yani bu turun anahtarına göre eşittirler. Yerleştirirken `cab` yine üstte, `fad` yine altta tutulur. Bu neden önemlidir? Çünkü en düşük anlamlı karakterlerde `b < d` olduğu için 1. turda `cab`, `fad`’in üstüne yerleşmişti. Son iki karaktere bakarsak gerçekten de “ab” < “ad” ’dir. Eşit anahtarlı elemanların birbirine göre sırası bozulmadığı sürece önceki turun bilgisi korunur.

> **Hocanın vurgusu:** `a`’lar birbirine eşit diye rastgele bir iş yapıp `fad`’i yukarı alsaydım, ilk basamakları sıralarken harcadığım bütün emek boşa gitmiş olacaktı. Stabil sıralamanın mantığı budur.

Soru: 2. ve 5. indislere baktığımızda (`ebb` ve `bad`) bunların yer değiştirdiği görülüyor. Bu kararlılığı bozmuyor mu?

Cevap: `ebb`’nin ikinci karakteri `b`, `bad`’in ikinci karakteri `a`’dır; yani bu turun anahtarına göre eşit değillerdir. Kararlılık yalnız eşit anahtarlı elemanlar için geçerlidir. `a < b` olduğu için `bad`’in zaten yukarı çıkması gerekir. Ama örneğin `eab` ile `fad` olsalardı, ikinci karakterleri (`a`) eşit olduğu için birbirlerine göre sıralarını bozmamak gerekirdi; yoksa son karakterlere göre

kurulan sıra bozulurdu. Eşitlerse göreli konum korunur, eşit değillerse duruma göre karar verilir.

Hoca aynı fikri sayılarla da anlattı: son basamakları eşit olan iki sayıda (ikisi de 9 ile biten) karar bir önceki basamağa göre verilir ve küçük olan yukarı çıkar.

3.6 3. Tur: İlk (1.) Karaktere Göre Sıralama

Son turda 2. turun sonucu üzerinde ilk karakterlere bakılır. Sayım:

Harf	a	b	c	d	e	f
Sayı	2	3	1	2	1	3
Başlangıç	0	2	5	6	8	9

Hoca bu turda `d`'nin başlangıcını önce 7 yazdı ve `dab`'ı 7. göze koydu. Yerleştirmeye devam ederken `ebb`'nin de 8'e geleceğini görünce tutarsızlığı fark etti; bir öğrencinin uyarısıyla `d`'den iki tane olduğu ve `d`'nin 6'dan başlaması gerektiği düzeltildi. Doğru iz:

Gelen dizgi	1. karakter	`tmp` gözü	Güncellenen `count`
dab	d	6	d: 6 → 7
cab	c	5	c: 5 → 6
fad	f	9	f: 9 → 10
bad	b	2	b: 2 → 3
dad	d	7	d: 7 → 8
ebb	e	8	e: 8 → 9
ace	a	0	a: 0 → 1
add	a	1	a: 1 → 2
fed	f	10	f: 10 → 11
bed	b	3	b: 3 → 4
fee	f	11	f: 11 → 12
bee	b	4	b: 4 → 5

Sonuç, başlangıçtaki dizinin küçükten büyüğe sıralanmış halidir:

```
indis:  0    1    2    3    4    5    6    7    8    9    10   11
        ace  add  bad  bed  bee  cab  dab  dad  ebb  fad  fed  fee
```

Dört aşamanın tamamı yan yana:

İndis	Başlangıç	3. karaktere göre	2. karaktere göre	1. karaktere göre
0	dab	dab	dab	ace
1	add	cab	cab	add
2	cab	ebb	fad	bad
3	fad	add	bad	bed
4	fee	fad	dad	bee
5	bad	bad	ebb	cab
6	dad	dad	ace	dab
7	bee	fed	add	dad
8	fed	bed	fed	ebb
9	bed	fee	bed	fad
10	ebb	bee	fee	fed
11	ace	ace	bee	fee

Bu turda da kararlılık işini görür: örneğin `dab` ile `dad`'in ilk karakterleri aynıdır (`d`); 2. turun sonunda `dab` üstte olduğu için son sıralamada da `dab` üstte kalır.

3.7 Hocanın İsim Örneği

Hoca kararlılığı bir isim listesiyle de anlattı. Bir kişinin adı “Elif”, soyadı “Karslıgil”; diğerinin adı “Elif”, soyadı “Sabun” olsun. Önce soyada göre sıralandığında “Karslıgil” “Sabun”un üstünde kalır. Sonra ada göre sıralanırken iki ad da “Elif” olduğu için eşittir. Bu durumda “Sabun”lu kaydı yukarı çıkarırsanız soyada göre yaptığınız sıralama boşa gider. Ama ikinci kişi “Ayşe Sabun” olsaydı, “Elif” “Ayşe” ’den büyük olduğu için “Ayşe Sabun”un yukarı çıkmasında bir sakınca olmazdı. Değerler eşit olduğunda önceki sıra bozulmamalıdır.

> **Ek bilgi (derste anlatılmadı):** N tane W karakterli dizgi ve R sembollük bir alfabe için LSD sıralama W tur anahtar indeksli sayma yapar. Her tur N elemanı ve $R+1$ gözlü `count` dizisini birkaç kez gezdiği için toplam süre $O(W(N+R))$, ek bellek ise `tmp` ve `count` için $O(N+R)$’dir. Yöntem hiç dizgi karşılaştırması yapmaz; karakterleri doğrudan indis olarak kullanır. Bu yüzden karşılaştırmalı sıralamalar için geçerli $O(N \log N)$ alt sınırına tabi değildir.

3.8 LSD Sıralamanın Kodu

Hoca kodu, anahtar indeksli sayma kodunu bütün karakter konumları için sağdan sola tekrarlayarak tahtada akıldan yazdı. Kodun parçaları şunlardır:

- Parametreler: dizgi dizisi `a`, dizgilerin karakter sayısı `w` ve sembol sayısı `R` (örneğin küçük harfler için 26).
- `tmp` için bellek ayrılır (hoca `malloc` kısmını yazmadı, “onu yazarsınız” dedi).
- Dış döngü `d = w - 1`’den başlar ve `d >= 0` olduğu sürece azalır. 379 örneğinde olduğu gibi önce son basamağa bakılır.
- Her turda `count` dizisi yeniden sıfırlanır. Sıfırlama döngüsünde koşul `r <= R` olmalıdır, çünkü `count` $R+1$ gözlüdür (sayılar bir göz kaydırılarak yazılır).
- Dizgi bir matris gibi düşünülür: `a[i]` dizgisinin `d`’inci karakteri `a[i][d]`’dir. Tek karakterli koddaki her `a[i] - 'a'` ifadesi `a[i][d] - 'a'` olur.
- Dizgiler `=` ile değil `strcpy` ile kopyalanır. Hoca, dağıtma adımında hesaplanan indisi `adr = a[i][d] - 'a'` diye bir değişkende tutmanın daha iyi olacağını söyledi. `tmp` de iki boyutlu (dizgi dizisi) olmalıdır.
- Her tur bitince `tmp` geri `a`’ya kopyalanır.

Hocanın tahtadaki hali altyazıdan şu şekilde kurulabildi:

```
void LSDSort(char a[], int w, int R)   /* tahtadaki hali; hatalı */
{
    int i, r, d;
    char *tmp;
    /* tmp için malloc */
    for (d = w - 1; d >= 0; d--) {
        for (r = 0; r <= R; r++)
            count[r] = 0;
        for (i = 0; i < w; i++)
            count[a[i][d] - 'a' + 1]++;
        for (r = 0; r < R; r++)
            count[r + 1] += count[r];
        for (i = 0; i < w; i++) {
            adr = a[i][d] - 'a';
            strcpy(tmp[i], a[i]);
            count[adr]++;
        }
        for (i = 0; i < w; i++)
            strcpy(a[i], tmp[i]);
    }
}
```

> **Ders sonrası not:** Hoca kodu akıldan yazdığını kendisi de söyledi; tahtadaki halde şu hatalar var. (1) `w` hem dizgi uzunluğu hem dizgi sayısı için kullanıldı. Dış döngü karakter konumlarını `d = W - 1`’den 0’a, iç döngüler ise dizgileri `i = 0`’dan `N - 1`’e gezmelidir; N dizgi sayısı, W dizgi uzunluğudur. (2) Dağıtma

adımında `strcpy(tmp[i], a[i])` yazıldı; doğrusu dizgiyi hesaplanan adrese koymaktır: `strcpy(tmp[count[adr]], a[i])`. (3) Parametre `char a[]` değil, iki boyutlu erişime izin veren `char a[][W + 1]` ya da `char **a` olmalıdır; `count` da $R+1$ gözlü tanımlanmalıdır.

Düzeltilmiş, derlenip çalıştırılmış tam program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define W 3   /* her dizginin karakter sayısı */

void LSDSort(char a[][W + 1], int N, int R)
{
    int i, r, d, adr;
    int *count = malloc((R + 1) * sizeof(int));
    char (*tmp)[W + 1] = malloc(N * sizeof *tmp);

    for (d = W - 1; d >= 0; d--) {          /* sağdan sola */
        for (r = 0; r <= R; r++)             /* her turda sıfırla */
            count[r] = 0;
        for (i = 0; i < N; i++)
            count[a[i][d] - 'a' + 1]++;
        for (r = 0; r < R; r++)
            count[r + 1] += count[r];
        for (i = 0; i < N; i++) {
            adr = a[i][d] - 'a';
            strcpy(tmp[count[adr]], a[i]);
            count[adr]++;
        }
        for (i = 0; i < N; i++)
            strcpy(a[i], tmp[i]);
    }
    free(count);
    free(tmp);
}

int main(void)
{
    char a[][W + 1] = {"dab", "add", "cab", "fad", "fee", "bad",
                       "dad", "bee", "fed", "bed", "ebb", "ace"};
    int i, N = sizeof(a) / sizeof(a[0]);
    LSDSort(a, N, 6);
    for (i = 0; i < N; i++)
        printf("%s ", a[i]);
    printf("\n");
    return 0;
}
```

Çıktı:

```
ace add bad bed bee cab dab dad ebb fad fed fee
```

Hoca kodu özetlerken şunu tekrarladı: önce `count` dizisine her harften kaç tane olduğu yazılır, sonra her harfin kaçıncı adresten başlayacağı bulunur (3 tane `a` varsa `b` 3. adresten, ardından 4 tane `b` varsa `c` 7. adresten başlar gibi), buna göre dizgiler geçici diziye alınır ve iş bitince `a` dizisine geri kopyalanır. LSD sıralama aynı işi bütün karakterler için, sağdan sola sırayla yapar.

4. Vize Sorularının Çözümü

Dersin ikinci yarısında hoca vize sorularını ekranda gösterdi ve üzerinde takılınan soruları çözdü. Soruların metni hocanın ekrandan okuduğu kadarıyla aşağıdadır; ekran görüntüsü olmadığı için seçenekli soruların seçenekleri her zaman bilinmiyor.

4.1 Soru 1: Yığın ile İkili Gösterim

Soru şöyleydi: Aşağıdaki yarı kod (pseudo code) parçası $n = 20$ değeri için ekrana XX yazar. Bu program YY işlemini gerçekleştirir.

Koddaki `push` yığına (stack) eleman yerleştirir, `pop` yığından eleman çeker. Kod, $n > 0$ olduğu sürece n mod 2 değerini yığına iter ve n'yi ikiye böler. Döngü bitince yığındaki her şeyi `pop` ederek yazdırır.

Hoca mantığı görmek için $n = 5$ ile elle çalıştırdı:

n	n mod 2 (push)	$n/2$
5	1	2
2	0	1
1	1	0

$n = 0$ olunca döngüden çıkılır. Yığına sırasıyla 1, 0, 1 itilmiştir; `pop` ile ters sırada çıkarılır ve ekrana `101` yazılır. Bu, 5'in ikili (binary) karşılığıdır. Kalanlar yığına itildiği için ters yüz edilmiş olarak, yani doğru sırada çıkar.

$n = 20$ için kalanlar sırasıyla 0, 0, 1, 0, 1 olur (ara adım eklendi) ve ekrana `10100` yazılır. Cevap: XX = `10100`, YY = bir sayının ikili karşılığını yazdırma.

4.2 Soru 2: Kuyruk ve Yığın

Soru: Aşağıdaki yarı kod parçasında kuyrukta (queue) başlangıçta sırasıyla 5, 2, 1, 4 sayıları bulunmaktadır. İşlem sonunda ekrana XXX yazılır.

Kod üç iş yapar:

- Kuyruk boşalana kadar kuyruktan eleman çeker ve yığına iter: 5, 2, 1, 4 sırasıyla yığına girer (en üstte 4).
- Yığın boşalana kadar yığından eleman çeker ve kuyruğa koyar: kuyruğa 4, 1, 2, 5 sırasıyla girer.
- Kuyruktaki elemanları sırayla çekip ekrana yazar.

Ekranda `4 1 2 5` görülür. Yani kuyruk, bir yığın yardımıyla ters çevrilmiştir.

4.3 Soru 3: Bağlı Listede `x->next = x->next->next`

Soru: Bir bağlı listedeki (linked list), son düğüm olmadığı bilinen bir `x` düğümü için `x->next = x->next->next` işlemi yapıldığında ne olur?

Cevap: `x->next`'in eskiden gösterdiği düğüme artık hiçbir işaretçi erişemez. Hoca "silinir" ile "erişilmez" ifadelerinin burada aynı anlama geldiğini, ayrıca o düğümün belleğinin `free` ile bırakılması gerektiğini söyledi. Örneğin listede $3 \to 4 \to 5$ varsa ve `x` 3'ü gösteriyorsa, bu işlemden sonra 3 doğrudan 5'i gösterir; aradaki 4'e kimse erişemez.

Soru: Ben cevapta işlemden sonra `x->next`'in nereyi göstereceğini yazmıştım. Bu doğru sayılmaz mı?

Cevap: `x->next`'in `x->next->next`'i göstereceği zaten sorunun içinde yazıyor; "3, 5'i gösterir" demek, yapılan işlemi tekrar etmektir. Soru, bunu yapınca ne olduğunu soruyor. Beklenen cevap, aradaki elemanı kimsenin göstermeyeceği, yani o elemanın erişilemez olacağıdır.

4.4 Soru 4: Yığın Ağacından En Büyük Elemanın Silinmesi

Soru: Aşağıdaki yığın ağacında (heap) en büyük elemanın silinip ağacın yeniden yığın haline getirilmesi (heapify) işleminde hangi sayılar karşılaştırma işlemine girer?

Hoca sınıfa ne olacağını sordu. Bir öğrenci 38 ile 19'un yer değiştireceğini, sonra daha büyüklerle değiştirileceğini söyledi; hoca bunu doğru buldu. En büyük eleman olan kökteki 38 silinir, ağacın son elemanı 19 köke gelir ve 19 aşağı taşınır. Her adımda 19'un iki çocuğu kendi arasında karşılaştırılır, büyük olan 19 ile karşılaştırılır ve 19'dan büyükse yer değiştirilir:

Adım	Karşılaştırılan çocuklar	Büyük olan	19 ile durum
1	37, 35	37	$37 > 19$, yer değiştir
2	34, 36	36	$36 > 19$, yer değiştir
3	14, 17	17	$17 < 19$, dur

19 her adımda karşılaştırmaya girdiği için cevap: 14, 17, 19, 34, 35, 36, 37.

> **Ders sonrası not:** Hoca 2. adımı anlatırken "38 onlardan hangisi büyükse oradan devam edecek" dedi. Aşağı taşınan sayı 38 değil, köke getirilen 19'dur; 38 ilk adımda silinmiştir.

4.5 Soru 5: Taban Koşulu Olmayan Özyineleme

Soru: Özyinelemeli (recursive) bir fonksiyonda taban koşulu (base condition) bulunmuyorsa aşağıdakilerden hangisi gerçekleşir?

Cevap: Sonsuz çalışma durumu oluşur. Taban koşulu yoksa durma şartı yoktur ve fonksiyon sürekli kendini çağırır.

4.6 Tek Bağlı Listeyle Kuyruk

Soru: Tek bağlı liste (singly linked list) kullanılarak oluşturulan bir kuyrukta ilk eklenen eleman için adres bilgisi saklanırken en son eklenen eleman için adres bilgisi ekstra olarak saklanmamaktadır. En kötü durumda zaman karmaşıklığı `enqueue` için XX, `dequeue` için YY'dir.

Cevap: `enqueue` $O(n)$, `dequeue` $O(1)$. Yeni eleman kuyruğun sonuna eklenir; sonun adresi tutulmadığı için her eklemede baştan sona gitmek gerekir. Çıkarma baştan yapılır ve başın adresi elde olduğu için sabit zamanda biter.

4.7 Bütün Elemanları Aynı Olan Diziyi Sıralamak

İfade: Tamamı aynı sayıdan oluşan n elemanlı bir dizi selection sort ile $O(n^2)$, insertion sort ile $O(n)$ karmaşıklıkla sıralanır.

Açıklama: Selection sort'un çaresi yoktur; dizi ne olursa olsun her adımda kalan kısmın tamamını tarar ve sıralamaya devam eder. Insertion sort ise karşılaştırmanın sonucuna göre bütün diziye bakmaya gerek duymaz: her eleman solundakinden küçük olmadığı için tek karşılaştırmada yerinde kalır. Python ile yapılan sayımda $n = 1000$ için selection sort $\frac{n(n-1)}{2} = 499500$, insertion sort $n - 1 = 999$ karşılaştırma yapar.

4.8 Dairesel Bağlı Listenin Önüne Eleman Eklemek

İfade: Dairesel bağlı liste (circular linked list) veri yapısının önüne yeni bir eleman eklemenin maliyeti en kötü durumda $O(1)$'dir. Hoca bu ifadeyi doğru olarak okudu.

> **Ders sonrası not:** $O(1)$ sonucu, listenin son düğümünü gösteren bir işaretçi tutuluyorsa geçerlidir: yeni düğüm son düğümün arkasına bağlanır ve son düğümün `next` alanı yeni düğümü gösterir. Yalnız baş işaretçisi tutuluyorsa son düğümün `next` alanını güncellemek için sona kadar gitmek gerekir ve maliyet $O(n)$ olur.

4.9 İkili Arama Ağacına Eleman Eklemek

Hoca ifadeyi "İkili arama ağacına (BST, binary search tree) eleman ekleme en kötü durumda büyük O notasyonunda $O(1)$'dir" diye okudu ve hemen nedenini açıkladı: en kötü durumda ağacın dengesi tamamen bozulmuştur, bütün elemanlar tek bir kol halinde dizilmiştir. Bu durumda n elemanınız varsa ararken hepsine bakarsınız ve karmaşıklık $O(n)$ olur. Ağaç dengeli olsaydı, bir tarafa ilerlediğinizde diğer tarafa bakmayacağınız için karmaşıklık $O(\log n)$ olurdu.

> **Ders sonrası not:** Hocanın açıklamasına göre BST'ye eklemenin en kötü durumu $O(n)$'dir; okunan ifadedeki $O(1)$ bu yüzden yanlıştır.

4.10 Yığın Çıktısı Soruları

Hoca bu grubu çözmeyeceğini, hepsinin `push/pop` işlemlerini doğru yapıp yapamadığınızı ölçtüğünü söyledi, yalnız iki örneğin cevabını okudu. Koddaki `for` döngüsü değerleri artan sırada sayarak yığına iter, `while` döngüsü ise sayıları ters sırada çıkarır.

- `1 3 5 2 4` çıktısı oluşabilir mi? Hayır. `1 3 5` oluşabilir, ama 5 çıkarıldığında yığında 2 ve 4 kalmıştır ve en üstte 4 vardır; 2, 4'ten önce çıkamaz.
- `1 3 5 4 2` çıktısı oluşabilir mi? Evet.

Hoca, bu tür soruları sayı vererek, yani işlemleri adım adım uygulayarak çözebileceğinizi söyledi. İki sonuç da 1'den 5'e kadar sayılar için bütün `push/pop` sıraları python ile denenerek doğrulandı.

4.11 Soru 12: İki Dizinin Ortak Elemanları

Hoca bunun çok hata yapılan bir soru olduğunu söyledi.

Soru: Sırasız bir A dizisi (N eleman) ve sıralı bir B dizisi (M eleman) veriliyor. N değerinin M değerinden çok daha büyük olduğu biliniyor. Bu iki dizideki ortak elemanları bulmak için nasıl bir algoritma tasarlarsınız? Bir cümleyle açıklayınız.

Cevap: A'daki her değer için B'de ikili arama (binary search) yapılır. Her arama $O(\log M)$ sürer; N arama yapıldığı için toplam karmaşıklık $O(N \log M)$ olur.

Hoca yanlış yaklaşımı da açıkladı. Öğrencilerin çoğu önce A'yı sıralayıp sonra arama yapmayı önermişti. Ama A'yı sıralamak tek başına $O(N \log N)$ tutar ve $N \gg M$ olduğu için bu, $O(N \log M)$'den pahalıdır. Hoca, "N değeri M'den çok büyük" denince öğrencilerin büyük olasılıkla "A'nın sırasız olmasındansa sıralasak daha iyi olur" diye düşündüklerini söyledi. Zaten işin içinde bir N olacaktır; onu yapmak yerine sıralı olan B üzerinde A'nın elemanları aranmalıdır.

> **Ek bilgi (derste anlatılmadı):** B'nin elemanları bir özet tablosuna (hash table) konursa her arama ortalama $O(1)$ sürer ve ortak elemanlar ortalama $O(N + M)$ sürede bulunabilir. Bu yol ek bellek ister ve en kötü durum garantisi vermez; sorudaki B'nin sıralı verilmesi, beklenen cevabın ikili arama olduğunu gösterir.

4.12 Soru 13 ve Devamı: Harflerle Maskelenmiş İkili Arama Ağacı

Soru: Aşağıdaki, sayı değerleri harflerle maskelenmiş olarak verilen ağacın bir ikili arama ağacı olduğu biliniyor. Sıralamada A düğümünden büyük ilk değer hangi düğümde yer alır?

Hoca, aynı ağaç üzerinde birkaç sorunun birleştirildiğini söyledi. Ağacın kökü A'dır; kökün sol alt ağacı B ile, sağ alt ağacı C ile başlar. Ağaçta A'dan M'ye 13 düğüm vardır.

- **A'dan büyük ilk değer: J.** A'dan büyük değerlerin hepsi sağ alt ağaçtadır; B ile başlayan sol alt ağaca hiç bakılmaz. C ile başlayan sağ alt ağaçta en soldaki, en alttaki düğüm A'dan büyük ilk değerdir. G, K, L ve M, C'den büyüktür, elenir. F, C'den küçüktür; J de F'den küçüktür ve daha solda düğüm yoktur. Bu yüzden cevap J'dir.
- **B'den büyük ilk değer: E.**
- **C'den büyük ilk değer: K.**
- **Ağacın yüksekliği: 4.** Kökün seviyesi 0 kabul edilip M'ye kadar inilir: 0, 1, 2, 3, 4.

- **Mevcut yüksekliği bozmadan eklenebilecek eleman sayısı: 18.** Yüksekliği 4 olan dolu bir ikili ağaçta $2^{4+1} - 1 = 31$ düğüm vardır (ara adım eklendi). Ağaçta 13 düğüm olduğu için $31 - 13 = 18$ eleman daha eklenebilir.

  Ek bilgi (derste anlatılmadı): İkili arama ağacında bir düğümden büyük ilk değere sıralı ardıl (inorder successor) denir. Düğümün sağ alt ağacı varsa ardıl, o sağ alt ağacın en soldaki düğümüdür; hocanın J için kullandığı kural budur. Sağ alt ağaç yoksa ardıl, düğümün sol alt ağacında bulunduğu en yakın atadır.

5. Programlama Sorusu: Arama Motoru Bağlı Listesi ve Tasarım Hataları

5.1 Sorunun Kuralları

Soru şöyleydi: Bir arama motorunda ziyaret edilen sayfalar, kod numarasına ve sayfaya bakılma sayısına göre aşağıdaki kurallara uygun olacak şekilde bağlı listeye yerleştirilmektedir:

- Bir sayfaya ilk defa bakıldıysa sayfanın kod numarası bağlı listenin başına yerleştirilir ve sayacı 1 olur.
- Bu eleman eklenince bağlı listedeki eleman sayısı N sınırını aştıysa, en sondaki sayfaya ait kod numarası listeden silinir.
- Sayfaya daha önce bakıldıysa sayacı artırılır.
- Sayaç değeri K'ya ulaştıysa o sayfaya ait kod numarası listenin başına getirilir ve sayacı sıfırlanır.

Hoca kuralları bir örnekle canlandırdı ($K = 4$):

Gelen sayfa	Liste (kod(sayaç)), baştan sona	Açıklama
3	3(1)	ilk kez geldi, başa eklendi
5	5(1) 3(1)	ilk kez geldi, başa eklendi
3	5(1) 3(2)	daha önce gelmiş, sayaç arttı
3	5(1) 3(3)	sayaç arttı
3	3(0) 5(1)	sayaç 4 = K oldu; başa alındı, sayaç sıfırlandı

5.2 Doğru Tasarım: Listeyi Bir Kez Gezmek

Hoca, mantıklı olan çözümün çok basit olduğunu ve sorunun zor bir soru olmadığını söyledi. Gelen kod numarası listede aranır; bulunursa sayacı artırılır, bulunamazsa başa eklenir. Önemli olan, bu işin listeyi tek bir kez gezerek yapılmasıdır:

- Eleman aranırken liste baştan gezilir.
- Aynı gezinti sırasında listedeki eleman sayısı da sayılır.
- Eleman bulunamazsa sona kadar gelinmiş demektir; o zaman eleman sayısı da bilinir.
- Son düğüm silinecekse, onu gösteren düğüm (sondan bir önceki) de bu gezinti sırasında tutulmuş olmalıdır.

5.3 Puan Kırılan Tasarım Hataları

Hoca, birçok cevapta tekrarlanan hatalı tasarımları tek tek anlattı:

- **Önce eklemek, sonra aramak.** Gelen 5'i önce listenin başına ekleyip, sonra listede 5'i aramak; bulursa eski 5'in sayacını 2 yapıp baştaki 5'i silmek. Doğrusu önce aramak, bulunursa artırmak, bulunamazsa eklemektir.
- **Aynı listeyi ikinci kez saymak.** Gelen 9'u aramak için sona kadar gidip bulamayınca başa eklemek, sonra listede kaç eleman olduğunu bulmak için en baştan bir daha saymak. Sona kadar zaten gelinmiştir; eleman sayısının bilinmesi gerekir.
- **Üçüncü bir tur atmak.** Bir turda 9'u aramak, ikinci turda eleman sayısının N'yi geçtiğini anlamak, üçüncü turda da silinecek son düğümü kimin gösterdiğini bulmak. Hoca bunun akıl alır bir şey olmadığını söyledi; son düğümü gösteren düğüm ilk turda tutulmalıydı.
- N **sınırını hiç kontrol etmemek.** Elemanı doğru yerleştirip K'ya göre sıfırlayıp başa almak, ama eleman sayısını kontrol etmeden uç uca eklemeye devam etmek.

- **Varlık kontrolü yapmamak.** Gelen her elemanı başa eklemek; örneğin 2 geldiğinde hem başa eklemek hem de listenin içinde zaten bir 2 bulunması.

  Hocanın vurgusu: Kod çalışsa da bu tasarımlar doğru kabul edilmez. İkinci sınıfın sonunda artık bunu yapmamamız gerekir. Çok ufak, etkinliği az etkileyen şeylere takılmıyoruz; ama karmaşıklıktan bahsederken n defa yapılan bir işi tekrar tekrar yapmak büyük bir hatadır.

Hoca ince çizgiyi de açıkladı: gereksiz yere bir şeyi tekrar tekrar tanımlamak ya da döngünün içinde bir değişkene sürekli aynı değeri atamak yapılmaması gereken ama küçük şeylerdir. Ama 100 eleman için her seferinde bütün listeyi yeniden gezmek, tasarımın kendisinin yanlış olduğunu gösterir. Aynı kodu iki kez yazdığınızı fark ettiğinizde tasarımınızı yeniden düşünmelisiniz.

5.4 Soru-Cevap: Program mı, Fonksiyon mu?

Soru: O soruyu cevaplarken `main`'i de tasarlamamız gerekiyor muydu? Örneğin kullanıcıdan sürekli giriş alıp bir tuşa basınca çıkmak gibi?

Cevap: Soruda tam program yazılması istendiği için `main` olmalıdır. Size "fonksiyon yazın" denirse fonksiyon yazarsınız, "program yazın" denirse `main` ile birlikte tam program yazmanız gerekir. Ama menü yapmaya ("1'e basarsan ekler, 2'ye basarsan listeler" gibi) ya da "çıkış tuşuna basılmadığı sürece" türü döngülere gerek yoktu. Çok basitçe on eleman ekleyeceğim deyip on giriş alabilir ya da kaç eleman ekleneceğini sorup onu okuyabilirsiniz.

Soru: Bilgilendirici mesajlara yer vermeli miyiz?

Cevap: Sonucu mutlaka yazdırmalısınız. Soru bu işi yapan algoritmayı tasarlamanızı istiyor; eleman eklendikten sonra listenin ne hale geldiği ekrana yazdırılmalıdır. Ama her şeyi incik cincik yazdırmanız gerekmez. Neyi okuyacağınızı ve neyi yazdıracağınızı biliyor olmanız gerekir; kod okunurken gereğinden fazla şey okunup okunmadığı ve doğru şeyin yazdırılıp yazdırılmadığı da değerlendirilir.

Hoca bu soruyla ilgili genel bir gözlem de paylaştı: birinci sınıfta kod yazmaya yeni başlarken öğrencilerin en büyük beklentisi kodun çalışmasıdır ve o aşamada bu yeterlidir. Ama ikinci sınıfın ikinci döneminde tasarımsal şeyleri düzgün yapabiliyor olmak gerekir; üçüncü sınıfta buna daha da çok dikkat edilir.

Örnek Sorular

Bu bölümdeki sorular derste sorulan ya da vize sorusu olarak ekranda gösterilip hocanın sesli okuduğu sorulardır. Bu ders için ekran karesi olmadığından soru metinleri altyazıdan alındı; bu yüzden hepsi "derste sözlü" olarak işaretlendi.

Soru 1 (derste sözlü, 00:10:10)

"kalem" ile "kel" dizgileri karşılaştırıldığında hangisi küçüktür? Neden?

Çözüm: İlk karakterler (`k`) eşittir. İkinci karakterde `a < e` olduğu için karşılaştırma burada biter: "kalem" < "kel". Dizgiler bir bütün olarak değil, ilk farklı karaktere kadar karakter karakter karşılaştırılır (Bölüm 1.1).

Soru 2 (derste sözlü, 00:44:15)

Sıralama yaparken işlerin yedek bir dizi üzerinden yürütülmesi, etkinlik açısından iyi mi, kötü mü?

Çözüm: Yedek dizi zaman kazandırır ama bellek harcar. Eleman sayısı küçükken (bin, on bin) sorun olmaz; eleman sayısı çok büyükse yedek dizi için ayrılan alan sorun yaratabilir. Bellek önemliyse dizinin kendisi üzerinde çalışan algoritmalar (selection, insertion, quicksort) tercih edilir (Bölüm 2.9).

Soru 3 (derste sözlü, 01:45:28)

Aşağıdaki yarı kod parçası $n = 20$ değeri için ekrana XX yazar. Bu program YY işlemini gerçekleştirir. (Kod: $n > 0$ olduğu sürece n mod 2 yığına itilir ve n ikiye bölünür; sonra yığın boşaltılarak her eleman yazdırılır.)

Çözüm: XX = `10100`, YY = sayının ikili karşılığını yazdırma. $n = 5$ için yığına 1, 0, 1 itilir ve `101` yazılır (Bölüm 4.1).

Soru 4 (derste sözlü, 01:48:19)

Kuyrukta başlangıçta sırasıyla 5, 2, 1, 4 vardır. Kuyruk boşalana kadar elemanlar çekilip yığına itilir; yığın boşalana kadar elemanlar çekilip kuyruğa konur; sonra kuyruktakiler sırayla yazdırılır. Ekrana ne yazılır?

Çözüm: `4 1 2 5` (Bölüm 4.2).

Soru 5 (derste sözlü, 01:50:03)

Bir bağlı listedeki son düğüm olmadığı bilinen bir `x` düğümü için `x->next = x->next->next` işlemi yapıldığında ne olur?

Çözüm: `x->next`'in eskiden gösterdiği düğüme artık erişilemez; o düğümün belleği `free` ile bırakılmalıdır (Bölüm 4.3).

Soru 6 (derste sözlü, 01:52:28)

Verilen maks-yığın ağacında en büyük elemanın silinip ağacın yeniden yığın haline getirilmesi işleminde hangi sayılar karşılaştırma işlemine girer?

Çözüm: 38 silinir, 19 köke gelir. 37-35 karşılaştırılır (37 yukarı çıkar), 34-36 karşılaştırılır (36 yukarı çıkar), 14-17 karşılaştırılır ve $17 < 19$ olduğu için durulur. Cevap: 14, 17, 19, 34, 35, 36, 37 (Bölüm 4.4).

Soru 7 (derste sözlü, 01:54:58)

Özyinelemeli bir fonksiyonda taban koşulu bulunmuyorsa aşağıdakilerden hangisi gerçekleşir?

Çözüm: Sonsuz çalışma durumu oluşur; durma şartı yoktur (Bölüm 4.5).

Soru 8 (derste sözlü, 01:55:17)

Tek bağlı liste ile kurulan bir kuyrukta ilk eklenen elemanın adresi saklanıyor, son eklenen elemanın adresi saklanmıyor. En kötü durumda `enqueue` ve `dequeue` için zaman karmaşıklıkları nedir?

Çözüm: `enqueue` $O(n)$, `dequeue` $O(1)$ (Bölüm 4.6).

Soru 9 (derste sözlü, 01:56:09)

Tamamı aynı sayıdan oluşan n elemanlı bir dizi selection sort ve insertion sort ile hangi karmaşıklıkla sıralanır?

Çözüm: Selection sort ile $O(n^2)$, insertion sort ile $O(n)$. $n = 1000$ için karşılaştırma sayıları 499500 ve 999'dur (Bölüm 4.7).

Soru 10 (derste sözlü, 01:56:34)

Dairesel bağlı listenin önüne yeni bir eleman eklemenin en kötü durumdaki maliyeti nedir?

Çözüm: Hoca $O(1)$ olarak okudu. Bu, son düğümü gösteren bir işaretçi tutulduğunda geçerlidir; yalnız baş işaretçisi varsa $O(n)$ olur (Bölüm 4.8).

Soru 11 (derste sözlü, 01:56:46)

İkili arama ağacına eleman eklemenin en kötü durumdaki karmaşıklığı nedir?

Çözüm: $O(n)$. Ağaç tamamen dengesiz, tek kol halindeyse bütün düğümler gezilir. Dengeli ağaçta $O(\log n)$ olur (Bölüm 4.9).

Soru 12 (derste sözlü, 01:57:33)

`push` işlemleri 1'den başlayarak artan sırada, `pop` işlemleri araya girerek yapılıyor. (a) `1 3 5 2 4` çıktısı oluşabilir mi? (b) `1 3 5 4 2` çıktısı oluşabilir mi?

Çözüm: (a) Hayır: 5 çıktıktan sonra yığında üstte 4, altta 2 vardır; 2, 4'ten önce çıkamaz. (b) Evet. İki sonuç da 1'den 5'e bütün geçerli `push`/`pop` sıraları python ile denenerek doğrulandı (Bölüm 4.10).

Soru 13 (derste sözlü, 01:58:05)

Sırasız bir A dizisi (N eleman) ve sıralı bir B dizisi (M eleman) veriliyor, $N \gg M$. İki dizideki ortak elemanları bulmak için nasıl bir algoritma tasarlarsınız? Bir cümleyle açıklayınız.

Çözüm: A'daki her eleman için B'de ikili arama yapılır; toplam $O(N \log M)$. Önce A'yı sıralamak tek başına $O(N \log N)$ tutar ve daha pahalıdır (Bölüm 4.11).

Soru 14 (derste sözlü, 01:59:50)

Sayı değerleri harflerle (A-M) maskelenmiş bir ikili arama ağacı veriliyor. (a) A'dan büyük ilk değer hangi düğümdedir? (b) B'den büyük ilk değer? (c) C'den büyük ilk değer? (d) Ağacın yüksekliği nedir? (e) Mevcut yüksekliği bozmadan kaç eleman daha eklenebilir?

Çözüm: (a) J (sağ alt ağacın en soldaki düğümü). (b) E. (c) K. (d) 4. (e) $2^5 - 1 - 13 = 18$ (Bölüm 4.12).

Soru 15 (derste sözlü, 02:02:11)

Bir arama motorunda ziyaret edilen sayfalar bağlı listede tutuluyor. İlk kez bakılan sayfanın kod numarası listenin başına sayacı 1 olarak eklenir; ekleme sonrası eleman sayısı N'yi aşarsa en sondaki sayfa silinir. Daha önce bakılmış sayfanın sayacı artırılır; sayaç K'ya ulaşırsa sayfa listenin başına alınır ve sayacı sıfırlanır. Bu işi yapan programı yazınız.

Hoca doğru tasarımı sözlü anlattı (listeyi bir kez gez; ararken say ve sondan bir önceki düğümü tut), ama kod yazmadı.

> **Çözüm (derste verilmedi):** Aşağıdaki program, hocanın anlattığı tek geçişli tasarıma göre yazıldı ve `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı. Giriş olarak N, K, gelecek sayfa sayısı ve sayfa kodları okunur; her ziyaretten sonra liste `kod(sayaç)` biçiminde yazdırılır.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int kod;              /* sayfa kod numarası */
    int sayac;            /* bakılma sayacı */
    struct node *next;
} NODE;

/* Listeyi tek kez gezer: arar, sayar, son düğümü ve öncesini tutar. */
NODE *ziyaret(NODE *head, int kod, int N, int K)
{
    NODE *p = head, *onceki = NULL, *oncekininOncesi = NULL, *yeni;
    int adet = 0;
```

```
    while (p != NULL && p->kod != kod) {
        oncekininOncesi = onceki;
        onceki = p;
        p = p->next;
        adet++;
    }

    if (p != NULL) {                        /* daha önce bakılmış */
        p->sayac++;
        if (p->sayac == K) {                /* K'ya ulaştı: başa al, sıfırla */
            p->sayac = 0;
            if (onceki != NULL) {
                onceki->next = p->next;
                p->next = head;
                head = p;
            }
        }
        return head;
    }

    /* ilk kez bakılıyor: liste sonuna kadar gezildi, adet = eleman sayısı */
    yeni = malloc(sizeof(NODE));
    yeni->kod = kod;
    yeni->sayac = 1;
    yeni->next = head;
    head = yeni;
    adet++;

    if (adet > N) {                         /* sınır aşıldı: sondakini sil */
        if (oncekininOncesi != NULL)
            oncekininOncesi->next = NULL;
        else
            yeni->next = NULL;
        free(onceki);
    }
    return head;
}

void yazdir(NODE *head)
{
    for (; head != NULL; head = head->next)
        printf("%d(%d) ", head->kod, head->sayac);
    printf("\n");
}

int main(void)
{
    NODE *head = NULL, *p;
    int N, K, M, i, kod;

    printf("N, K ve sayfa sayisi: ");
    if (scanf("%d %d %d", &N, &K, &M) != 3)
        return 1;
    for (i = 0; i < M; i++) {
        if (scanf("%d", &kod) != 1)
            break;
        head = ziyaret(head, kod, N, K);
        yazdir(head);
    }
    while (head != NULL) {
```

```
        p = head;
        head = head->next;
        free(p);
    }
    return 0;
}
```

Hocanın örneği ($K = 4$, sayfalar 3, 5, 3, 3, 3; liste sınırı aşılmadığı için $N = 3$ seçildi) ile çıktı (giriş istemi satırı hariç):

```
3(1)
5(1) 3(1)
5(1) 3(2)
5(1) 3(3)
3(0) 5(1)
```

N sınırının çalıştığını görmek için yapılan deneme ($N = 3$, $K = 4$, sayfalar 1, 2, 3, 4, 2, 5) çıktısı:

```
1(1)
2(1) 1(1)
3(1) 2(1) 1(1)
4(1) 3(1) 2(1)
4(1) 3(1) 2(2)
5(1) 4(1) 3(1)
```

4 geldiğinde eleman sayısı 4 olur ve sondaki 1 silinir; 5 geldiğinde sondaki 2 silinir. Döngü bulunamayan eleman için listeyi bir kez sona kadar gezer; bu sırada `adet` eleman sayısını, `onceki` son düğümü, `oncekininOncesi` sondan bir önceki düğümü tutar. Böylece ne ikinci bir sayma turuna ne de silinecek düğümün öncesini bulmak için üçüncü bir tura gerek kalır.

Dersin Toparlaması

Bu derste dizgilerin sayılardan farklı olarak karakter karakter karşılaştırıldığını gördük ve dizgi sıralamanın yapı taşı olan anahtar indeksli saymayı öğrendik. Anahtar indeksli sayma, sembolleri doğrudan dizi indisi olarak kullanır: önce her sembolün frekansı bir kaydırılmış gözlere sayılır, kümülatif toplamla her sembolün sıralı dizideki başlangıç indisi bulunur, elemanlar bu indislere göre geçici diziye dağıtılır ve geri kopyalanır. `count` dizisinin sıfırlanması şarttır. Yöntem hızlıdır ama yedek dizi kullandığı için bellek harcar; bellek önemliyse bu bir dezavantajdır. LSD dizgi sıralama, aynı işlemi sabit uzunluklu dizgilerin son karakterinden ilk karakterine kadar her konum için tekrarlar. Yöntemin doğru çalışması kararlılığa dayanır: eşit anahtarlı elemanların önceki turdaki göreli sırası korunmazsa, önceki turlarda yapılan sıralama boşa gider. MSD dizgi sıralama öğrencilere inceleme için bırakıldı.

Vize sorularının çözümünde yığın, kuyruk, bağlı liste, yığın ağacı, özyineleme, ikili arama ağacı ve sıralama karmaşıklıkları tekrar edildi. En çok hata yapılan iki noktada hocanın mesajı aynıydı: karmaşıklığı düşünerek tasarım yapmak. Ortak eleman sorusunda büyük diziyi sıralamak yerine küçük ve sıralı dizide ikili arama yapmak gerekir; bağlı liste sorusunda ise listeyi bir kez gezerek arama, sayma ve silme hazırlığını birlikte yapmak gerekir. Hoca, sınavdan çıkar çıkmaz soruları kaynaklar açıkken yeniden çözmeyi ve yanlışları böyle öğrenmeyi önerdi. Veri yapıları ve algoritmaların iyi bir bilgisayar mühendisi olmak için en temel derslerden biri olduğunu hatırlatarak, yaz boyunca bol bol farklı algoritma problemi çözmeyi tavsiye etti: önce derste anlatılan sık kullanılan algoritmalar iyice öğrenilirse, o tarzlar üzerinden çok farklı problemler de iyi bir şekilde çözülebilir hale gelir. Bu konuların devamı gelecek dönem Algoritma Analizi dersinde (örneğin merge sort ve quicksort'un analizi) görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.